TECHNICAL ASSIGNMENT

mine MARKETING

The Agent Army Challenge

Technical Data & Automation Specialist

Background

You're joining a performance marketing company that runs large-scale paid campaigns on Meta. Today, a team of media buyers makes dozens of daily decisions: pause/scale adsets, adjust budgets, change bids, kill losers, duplicate winners. We also run **auto-rules** - simple threshold-based automations (e.g., "pause adset if spend > $50 and ROI < 0.05").

The media buyers are expensive. Your mission: **design and prove out an AI agent system that can take over most of the media buyers' decision-making - profitably, safely, and cost-effectively.**

The mandate is to **maintain or grow total spend and absolute profit while improving efficiency** - not simply to maximize ROI by shrinking. An agent that pauses everything would score a great ROI and destroy the business.

We want to see how you think.

What you get

A set of table snapshots (CSV) representing **one week** of activity across **six Meta ad accounts** (`ACC-01` … `ACC-06`), plus a `README_DATA.md` with column documentation:

File	Contents
`campaign_adset_metadata.csv`	Campaign/adset configuration: status, budgets, bid strategy, targeting, naming conventions
`daily_adset_performance.csv`	Daily adset performance: spend, impressions, clicks, conversions, revenue, ROI and derived metrics
`auto_rules.csv`	The 12 active auto-rule definitions (logic encoded in the rule name, as in the real system)
`rule_executions.csv`	Full log of rule-engine actions during the week: timestamps, budgets before/after, the metrics the engine saw at evaluation time, and the API response
`buyer_actions.csv`	Manual actions taken by media buyers: timestamp, adset, event type, budget change, optional free-text note

> ⚠ **This data comes from production-like systems. Like all production data, it is not guaranteed to be clean, consistent, or complete. Treat it the way you'd treat real data.**

Rules of engagement

- **You are required to use Claude Code** (or a comparable AI coding tool) throughout. We want to see how you leverage AI - this is a core skill for the role.
- **You are required to keep an AI usage log** (see Deliverable 4). Using AI is not cheating here. Using AI *blindly* is.

Task A - Investigate before you automate

Before designing agents, understand the battlefield. Last week, leadership suspects the **auto-rules destroyed money** - killing winners and letting losers run.

1. Reconstruct what the auto-rules actually did over the week. Quantify their impact: how much money did rule-driven actions save or burn? Show your methodology - there is no single "correct" way to attribute this, so state your assumptions explicitly.
2. Find at least **two concrete cases** where a rule made a decision a competent human would not have made. Explain *why* the rule got it wrong - what information was it missing or misreading?
3. List any **data issues** you encountered and how you handled them. If you found none, say so - and tell us how you checked.

Deliverable: `INVESTIGATION.md` + the queries/scripts you used.

Task B - Design the agent army (no code required)

Design a POC architecture for an agent system that replaces the bulk of media buyer decision-making.

Your design doc must answer, at minimum:

1. **Agent topology.** One agent or many? If many - what are the roles (e.g., monitor, analyst, executor, auditor), and why this split? What does each agent see, and what can each agent *do*?
2. **Decision boundaries.** Which decisions can an agent take autonomously, which require human approval, and which are forbidden? Define concrete thresholds (e.g., max budget change per action, max daily total exposure per agent).
3. **The economics.** Estimate the LLM API cost of your system per day for these six ad accounts (show your math: how many decisions/day, tokens per decision, model choice per task). At

what account scale does this beat a media buyer's salary? Where would you use a cheap model vs. an expensive one - and where would you use **no model at all** (plain code/SQL)?

4. **Failure modes.** List your top 5 ways this system loses money or breaks, and the guardrail for each. Include the kill-switch design.
5. **Data flow.** The agents have access to all of the CSVs. Which data do you actually feed each agent, at what frequency, and how is it expected to read it? Walk us through the whole architecture: what gets pulled on every cycle vs. cached vs. queried on demand, how raw tables become decision-ready context, and how the `buyer_actions.csv` history factors in (including the risks of relying on it).

Constraints (hard):

- LLM API budget for the POC phase: **$30/day** across all six accounts.
- Revenue data arrives with delay (you'll discover the characteristics in Task A).
- Meta API rate limits exist; assume you can't poll everything every minute.

Deliverable: `ARCHITECTURE.md` (text + at least one diagram; ASCII/Mermaid is fine).

Task C - Build one working slice

Implement **one agent** from your architecture, end-to-end, running against the snapshot data: the **Adset Decision Agent**.

For each active adset, on each of the last 3 days in the data (so the agent has some history to work with), it must output:

```
{
  "adset_id": "...",
  "decision_date": "...",
  "action": "scale_up | scale_down | pause | keep | escalate",
  "amount": null | <new daily budget>,
  "confidence": 0.0-1.0,
  "reasoning": "...",
  "data_quality_flags": ["..."]
}
```

This is the minimum schema - add any fields you think make the agent's output more useful or auditable (e.g. expected profit impact, which data the decision relied on, assumptions made), and tell us why.

Requirements:

1. It must actually call an LLM for the judgment layer (Anthropic API; we'll reimburse up to $10 of API usage - and staying *under* that is part of the test).

2. It must **not** send raw full tables to the model. Show us how you compress/structure context per decision.
3. **The uncertainty question:** at least one situation in the data has no clean answer. Your agent must *recognize* when it doesn't know - not guess confidently. Show us the mechanism (this is the part we care about most).
4. **The improvement question:** propose and stub (interface/pseudocode is fine) the feedback loop - how does the agent get better next week than it is this week, *without* anyone retraining a model? What signal does it learn from, where is that signal stored, and how does it change future decisions?
5. Compare your agent's decisions to what the auto-rules and the human buyers actually did on the same days. Where do you disagree with the humans - and who do you think was right?
6. **The evaluation question:** how do you measure whether your agent actually did a good job? Define your success metric(s) and defend them - what do you look at, why that and not something else, and how do you handle the fact that the "right" answer for a given day isn't knowable until revenue finishes arriving? A strong answer here is worth more to us than a clever prompt.

Deliverable: Working code + `RESULTS.md` with the comparison and your honest assessment of where your agent is weak.

Task D - The AI usage log (ongoing)

Keep `DECISIONS.md` as you work. For each significant step, record briefly:

- What you asked Claude Code to do (key prompts, paraphrased is fine)
- What it got **wrong** or what you **rejected/changed**, and why
- Decisions you deliberately made yourself rather than delegating - and why

A log that says "Claude did everything and it was all correct" tells us either you didn't look closely, or you didn't push it hard enough. The most impressive logs we've seen are the ones showing a real argument between the candidate and the model.

Submission & review

Submit a Git repo (private GitHub link or zip) containing: code, `INVESTIGATION.md`, `ARCHITECTURE.md`, `RESULTS.md`, `DECISIONS.md`, and a short `README.md` (how to run, where you cut corners, total time spent).

Good luck - we're genuinely curious to see your approach.